

DASH-05: Engineering Dashboards

Series: DASH | **Notebook:** 5 of 7 | **Created:** March 2026 | **Last Updated:** 04/04/2026

Overview

Engineering dashboards are the deepest tier in the dashboard hierarchy. They serve developers, DBAs, and performance engineers who need to answer specific questions: Which endpoint is slow? Which database query is causing timeouts? Did the last deployment improve or degrade latency? This notebook covers trace analysis, database performance, endpoint-level metrics, deployment impact analysis, and effective use of filters and variables for drill-down workflows.

Table of Contents

- [1. Span Analysis for Service Deep-Dives](#)
 - [2. Database Performance](#)
 - [3. Endpoint-Level Metrics](#)
 - [4. Deployment Impact Analysis](#)
 - [5. Code-Level Metrics](#)
 - [6. Engineering Dashboard Layout](#)
 - [7. Summary and Next Steps](#)
-

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS or Managed with Grail enabled
Permissions	<code>storage:spans:read</code> , <code>storage:logs:read</code> , <code>storage:metrics:read</code>
Data	Distributed trace data (spans), database spans, deployment events
Prior Reading	DASH-01 through DASH-04

1. Span Analysis for Service Deep-Dives

Engineering dashboards rely heavily on span data. Spans provide the most granular view of service behavior — individual requests, their duration, status, and attributes.

Service Throughput and Latency

```
// Service throughput and latency – engineering overview
fetch spans, from:-1h
| filter span.kind == "server"
| summarize request_count = count(), avg_ms = avg(duration) / 1000000, p50_ms
= percentile(duration, 50) / 1000000, p95_ms = percentile(duration, 95) /
1000000, p99_ms = percentile(duration, 99) / 1000000, error_count =
countIf(otel.status_code == "ERROR"), by:{dt.entity.service}
| fieldsAdd error_rate_pct = round(100.0 * error_count / request_count,
decimals: 2)
| sort p95_ms desc
```

Error Span Details

When investigating errors, engineers need the actual span attributes — status code, error message, and trace ID for correlation.

```
// Recent error spans with details – engineering investigation table
fetch spans, from:-1h
| filter span.kind == "server" and otel.status_code == "ERROR"
| fieldsKeep timestamp, trace.id, dt.entity.service, http.route,
http.response.status_code, otel.status_message, duration
| fieldsAdd duration_ms = duration / 1000000
| fieldsRemove duration
| sort timestamp desc
| limit 25
```

Latency Distribution

Understanding the shape of latency distribution helps identify whether slowness is systemic or affects only a tail of requests.

```
// Latency distribution buckets – engineering histogram
fetch spans, from:-1h
| filter span.kind == "server"
| fieldsAdd duration_ms = duration / 1000000
| fieldsAdd latency_bucket = if(duration_ms < 100, then: "<100ms",
else: if(duration_ms < 500, then: "100-500ms", else: if(duration_ms <
1000, then: "500ms-1s", else: if(duration_ms < 5000, then: "1s-5s", else:
">5s"))))
| summarize request_count = count(), by:{latency_bucket}
| sort request_count desc
```

2. Database Performance

Database spans reveal query performance, connection issues, and which databases are bottlenecks.

Database Response Time by System

```
// Database response time by system and namespace
fetch spans, from:-1h
| filter span.kind == "client" and isNotNull(db.system)
| summarize avg_ms = avg(duration) / 1000000, p95_ms = percentile(duration,
95) / 1000000, query_count = count(), errors = countIf(otel.status_code ==
"ERROR"), by:{db.system, db.namespace}
| fieldsAdd error_rate = round(100.0 * errors / query_count, decimals: 2)
| sort p95_ms desc
```

Slowest Database Operations

```
// Slowest database operations – engineering deep-dive
fetch spans, from:-1h
| filter span.kind == "client" and isNotNull(db.system)
| summarize avg_ms = avg(duration) / 1000000, max_ms = max(duration) /
1000000, call_count = count(), by:{db.system, db.statement}
| sort max_ms desc
| limit 15
```

Database Query Trend

```
// Database query volume and latency trend
fetch spans, from:-2h
| filter span.kind == "client" and isNotNull(db.system)
| makeTimeseries avg_ms = avg(duration) / 1000000, query_count = count(),
interval:5m, by:{db.system}
```

3. Endpoint-Level Metrics

Endpoint-level analysis lets engineers identify which specific API routes are problematic.

Endpoint Performance Table

```
// Endpoint performance breakdown – engineering detail table
fetch spans, from:-1h
| filter span.kind == "server" and isNotNull(http.route)
| summarize requests = count(), avg_ms = avg(duration) / 1000000, p95_ms =
percentile(duration, 95) / 1000000, errors = countIf(otel.status_code ==
"ERROR"), by:{http.route, http.request.method}
| fieldsAdd error_rate = round(100.0 * errors / requests, decimals: 2)
| sort p95_ms desc
| limit 20
```

Endpoint Latency Trend

```
// Endpoint latency trend – line chart for top 5 routes
fetch spans, from:-2h
| filter span.kind == "server" and isNotNull(http.route)
| makeTimeseries p95_ms = percentile(duration, 95) / 1000000, interval:5m,
by:{http.route}
```

4. Deployment Impact Analysis

Compare service performance before and after a deployment to quantify its impact.

Before vs After Deployment

This pattern uses `append` to compare two time windows — 1 hour before the deployment vs 1 hour after.

```
// Before/after deployment comparison – last 2 hours vs preceding 2 hours
fetch spans, from:-2h
| filter span.kind == "server"
| summarize after_avg_ms = avg(duration) / 1000000, after_p95_ms =
percentile(duration, 95) / 1000000, after_error_rate = 100.0 *
countIf(otel.status_code == "ERROR") / count()
| append [
  fetch spans, from:-4h, to:-2h
  | filter span.kind == "server"
  | summarize before_avg_ms = avg(duration) / 1000000, before_p95_ms =
percentile(duration, 95) / 1000000, before_error_rate = 100.0 *
countIf(otel.status_code == "ERROR") / count()
]
```

5. Code-Level Metrics

When services are instrumented with custom spans, engineers can see method-level performance.

Span Performance by Code Namespace

```
// Code-level performance by namespace and function
fetch spans, from:-1h
| filter isNotNull(code.namespace)
| summarize avg_ms = avg(duration) / 1000000, call_count = count(), by:
{code.namespace, code.function}
| sort avg_ms desc
| limit 20
```

Span Hierarchy — Call Chain Analysis

```
// Span breakdown by kind – understanding the call chain
fetch spans, from:-1h
| summarize span_count = count(), avg_ms = avg(duration) / 1000000, by:
{span.kind}
| sort span_count desc
```

6. Engineering Dashboard Layout

Recommended Sections

Section	Tiles	Notes
Service Overview	Throughput table, error rate, latency percentiles	Use variables for service selection
Endpoint Detail	Endpoint performance table, latency trend	Filter by HTTP route
Database	DB response time table, query trend	Filter by db.system
Code Level	Namespace/function performance	Only if custom instrumentation exists
Deployment	Before/after comparison, recent events	Correlate with latency changes

Engineering Dashboard Best Practices

- **Use variables extensively** — service, namespace, and time range selectors let engineers focus on their area
- **Include trace links** — table tiles with `trace.id` let engineers click through to the full distributed trace
- **Short time ranges** — 15m to 1h for investigation; longer ranges hide the signal in noise
- **Table tiles over charts** — engineers often need exact values, not visual trends
- **Document queries** — add markdown tiles explaining what each section shows and how to interpret it

7. Summary and Next Steps

In this notebook you learned:

- Span-based service analysis with throughput, latency percentiles, and error rates
- Database performance monitoring using client spans
- Endpoint-level metrics for API route analysis

- Before/after deployment comparison patterns
- Code-level metrics for instrumented services
- Layout recommendations and best practices for engineering dashboards

Next: In **DASH-06: Variables and Filters**, we cover how to make dashboards dynamic and reusable with variables, filter propagation, and template patterns.

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.